Informe del Proyecto: Simulador del Algoritmo del Banquero

Universidad Mariano Gálvez de Guatemala — Facultad de Ingeniería · Sistemas Operativos 1

Estudiante: Luis Antonio Santos Rodriguez · Carné: [escribir aquí]

1. Descripción de la solución

El programa simula el Algoritmo del Banquero, que los sistemas operativos usan para asignar recursos de forma segura y evitar interbloqueos (deadlocks). El usuario ingresa el número de procesos (N), el número de tipos de recursos (M), las matrices Allocation y Max y el vector Available. El programa calcula automáticamente la matriz Need (Need = Max − Allocation), ejecuta el algoritmo de seguridad e indica si el sistema está en estado seguro o inseguro. Si el estado es seguro, muestra la secuencia segura con la que se pueden atender los procesos.

Estructuras de datos. Todas se representan con listas de Python (la fila i es el proceso Pi y la columna j es el recurso Rj):

- Available (1 × M): instancias libres de cada recurso.
- Max (N × M): demanda máxima de cada proceso.
- Allocation (N × M): recursos asignados actualmente a cada proceso.
- Need (N × M): recursos que aún puede pedir cada proceso (Max − Allocation).
- Work (1 × M): copia de trabajo de Available durante el algoritmo.
- Finish (1 × N): indica si el proceso ya pudo terminar en la simulación.

Algoritmo de seguridad. Se implementa en la función `algoritmo_seguridad`:

1. Work = Available y Finish[i] = falso para todos los procesos.
2. Se busca un proceso i con Finish[i] = falso y Need[i] ≤ Work.
3. Si existe, el proceso termina y devuelve sus recursos: Work = Work + Allocation[i], Finish[i] = verdadero y se agrega a la secuencia segura. Se vuelve al paso 2.
4. Si no existe ninguno, el estado es seguro cuando todos los Finish son verdadero, e inseguro (posible interbloqueo) en caso contrario.

Los procesos se recorren en rondas y el programa muestra el detalle de cada una: qué proceso se evalúa, cómo se actualiza Work y por qué un proceso termina o debe esperar. La complejidad del algoritmo es $O(M \cdot N^2)$.

Funcionalidades incluidas:

- Ingreso de datos con validación: números enteros, sin negativos, cantidad exacta de valores por fila y Allocation ⩽ Max.
- Cálculo y visualización automática de la matriz Need.
- Resultado explícito: estado seguro o inseguro, con la secuencia segura (o los procesos que quedan bloqueados).
- Escenarios de prueba precargados, que se pueden cargar sin reiniciar el programa.
- Simulación de solicitudes de recursos (Request): si el estado resultante es seguro la solicitud se concede y el sistema se actualiza; si no, se deniega.
- Menú interactivo, mensajes claros y manejo de errores de entrada.

El código fuente está en un solo archivo, `banquero.py`, organizado en funciones de entrada de datos, funciones del algoritmo (`calcular_need`, `algoritmo_seguridad`, `solicitar_recurso`), funciones de presentación y el menú principal (`main`). Cada función está explicada con comentarios.

2. Tecnología utilizada

- Lenguaje: Python 3 (versión 3.6 o superior).
- Bibliotecas: solo la biblioteca estándar (módulo `copy`); no hay que instalar paquetes adicionales.
- Interfaz: consola (menú de texto).
- Sistemas operativos: Linux, Windows y macOS.
- Herramientas: cualquier editor de texto o IDE para ver el código; para ejecutarlo basta una terminal.

3. Instrucciones de instalación y ejecución

1. Verifique que Python 3 está instalado escribiendo `python3 --version` en una terminal (en Windows: `python --version`). Si no lo está, descárguelo desde python.org (en Linux, instálelo con el gestor de paquetes de la distribución).
2. Guarde el archivo `banquero.py` en una carpeta.
3. Abra una terminal en esa carpeta.
4. Ejecute `python3 banquero.py` (en Windows: `python banquero.py` o `py banquero.py`).
5. Aparece el menú principal. Las instrucciones de uso están en el manual de usuario.

No se requiere instalar dependencias ni tener conexión a internet.

4. Casos de prueba

Los casos 1 y 2 se cargan con la opción 2 del menú (escenarios 1 y 2) y se ejecutan con la opción 4. El caso 3 usa la opción 5.

Caso 1: estado seguro

Sistema con 5 procesos y 3 recursos. Datos de entrada y matriz Need calculada por el programa:

```
Proceso  Allocation      Max             Need
           R0  R1  R2      R0  R1  R2      R0  R1  R2
P0          0   1   0       7   5   3       7   4   3
P1          2   0   0       3   2   2       1   2   2
P2          3   0   2       9   0   2       6   0   0
P3          2   1   1       2   2   2       0   1   1
P4          0   0   2       4   3   3       4   3   1

Available: R0=3  R1=3  R2=2
```

Resultado del programa:

```
Work inicial = Available = [3, 3, 2]

  Ronda 1  (Work = [3, 3, 2])
    P0: Need [7, 4, 3] NO es <= Work [3, 3, 2] -> espera
    P1: Need [1, 2, 2] <= Work [3, 3, 2] -> TERMINA y libera [2, 0, 0]
        Work = [3, 3, 2] + [2, 0, 0] = [5, 3, 2]
    P2: Need [6, 0, 0] NO es <= Work [5, 3, 2] -> espera
    P3: Need [0, 1, 1] <= Work [5, 3, 2] -> TERMINA y libera [2, 1, 1]
        Work = [5, 3, 2] + [2, 1, 1] = [7, 4, 3]
    P4: Need [4, 3, 1] <= Work [7, 4, 3] -> TERMINA y libera [0, 0, 2]
        Work = [7, 4, 3] + [0, 0, 2] = [7, 4, 5]

  Ronda 2  (Work = [7, 4, 5])
    P0: Need [7, 4, 3] <= Work [7, 4, 5] -> TERMINA y libera [0, 1, 0]
        Work = [7, 4, 5] + [0, 1, 0] = [7, 5, 5]
    P2: Need [6, 0, 0] <= Work [7, 5, 5] -> TERMINA y libera [3, 0, 2]
        Work = [7, 5, 5] + [3, 0, 2] = [10, 5, 7]

============================================================
  RESULTADO: ESTADO SEGURO
  Secuencia segura: P1 -> P3 -> P4 -> P0 -> P2
============================================================
```

Análisis: los cinco procesos logran terminar, por lo que el estado es seguro y la secuencia segura obtenida es P1 → P3 → P4 → P0 → P2. Como comprobación, el Work final [10, 5, 7] es igual a Available más la suma de la matriz Allocation, es decir, al total de instancias del sistema.

Caso 2: estado inseguro

Sistema con 4 procesos y 3 recursos. Datos de entrada y matriz Need calculada por el programa:

```
Proceso  Allocation      Max             Need
           R0  R1  R2      R0  R1  R2      R0  R1  R2
P0          1   0   0       2   0   1       1   0   1
P1          0   1   1       0   2   1       0   1   0
P2          1   1   0       3   2   0       2   1   0
P3          0   1   0       0   3   1       0   2   1

Available: R0=1  R1=0  R2=1
```

Resultado del programa:

```
Work inicial = Available = [1, 0, 1]

  Ronda 1  (Work = [1, 0, 1])
    P0: Need [1, 0, 1] <= Work [1, 0, 1] -> TERMINA y libera [1, 0, 0]
        Work = [1, 0, 1] + [1, 0, 0] = [2, 0, 1]
    P1: Need [0, 1, 0] NO es <= Work [2, 0, 1] -> espera
    P2: Need [2, 1, 0] NO es <= Work [2, 0, 1] -> espera
    P3: Need [0, 2, 1] NO es <= Work [2, 0, 1] -> espera

  Ronda 2  (Work = [2, 0, 1])
    P1: Need [0, 1, 0] NO es <= Work [2, 0, 1] -> espera
    P2: Need [2, 1, 0] NO es <= Work [2, 0, 1] -> espera
    P3: Need [0, 2, 1] NO es <= Work [2, 0, 1] -> espera

============================================================
  RESULTADO: ESTADO INSEGURO (posible interbloqueo)
  Procesos que NO pueden terminar: P1, P2, P3
  Procesos que sí alcanzaron a terminar: P0
============================================================
```

Análisis: solo P0 logra terminar. Al devolver sus recursos, Work queda en [2, 0, 1], sin ninguna instancia de R1. P1, P2 y P3 necesitan al menos una instancia más de R1 y las únicas instancias de R1 están retenidas por ellos mismos, así que ninguno puede avanzar.

En la ronda 2 nadie progresa y el programa concluye que el estado es inseguro (posible interbloqueo).

Caso 3 (adicional): solicitudes de recursos

Sobre el escenario 1 se simularon cuatro solicitudes seguidas con la opción 5. Respuestas del programa:

```
P1 solicita [1, 0, 2]
  SOLICITUD CONCEDIDA: el estado resultante es SEGURO.
  Secuencia segura: P1 -> P3 -> P4 -> P0 -> P2

P0 solicita [0, 2, 0]
  SOLICITUD DENEGADA: conceder [0, 2, 0] a P0 dejaría el sistema en ESTADO
INSEGURO.

P4 solicita [3, 3, 0]
  P4 DEBE ESPERAR: pide [3, 3, 0] pero solo hay disponible [2, 3, 0].

P2 solicita [9, 0, 0]
  SOLICITUD RECHAZADA: P2 pide [9, 0, 0] pero su Need es [6, 0, 0] (excede el
máximo que declaró).
```

Análisis: la primera solicitud se concede y el sistema se actualiza (Available pasa de [3, 3, 2] a [2, 3, 0]). La segunda se deniega aunque hay recursos libres, porque dejaría a todos los procesos sin poder avanzar. La tercera debe esperar porque pide más de lo disponible. La cuarta se rechaza porque excede el máximo declarado por el proceso.

5. Conclusiones

El simulador cumple con lo solicitado: ingresa y valida los datos, calcula la matriz Need, ejecuta el algoritmo de seguridad, determina si el estado es seguro o inseguro y muestra la secuencia segura. Los casos de prueba confirman que detecta correctamente ambas situaciones. Una misma configuración puede tener varias secuencias seguras; el programa muestra la primera que encuentra al recorrer los procesos en orden.